

FACTORY METHOD FIGURAS

Patrón de Diseño: *Factory Method*

Cálculo de áreas de figuras geométricas usando
el patrón Factory Method

Autor: **Angie Nathaly Cisneros Guerrrero**

20242020084

Fecha: Agosto 2026

Índice

1. Diagrama UML	2
2. Explicación de las Clases	3
2.1. Producto - Figura	3
2.2. Productos Concretos de Figuras	4
2.2.1. Clase Círculo	4
2.2.2. Clase Rectángulo	4
2.2.3. Clase Triángulo	5
2.3. Interfaz Factory - FabricaFigura	7
2.3.1. FabricaCírculo	8
2.3.2. FabricaRectangulo	8
2.3.3. FabricaTriangulo	9
3. Código de Clientes y Explicación	10
3.1. Clase Cliente	10
3.2. Main	11
4. Pruebas	13
4.1. Prueba de Círculo	13
4.2. Prueba de Rectángulo	13
4.3. Prueba de Triángulo	15
5. Código	16
5.1. Link Github	16
5.2. Paquetes	16

Diagrama UML

Descripción General

El proyecto Factory Method Figures implementa el patrón de diseño **Factory Method** para la creación de objetos que representan figuras geométricas (Círculo, Rectángulo y Triángulo). Este patrón proporciona una interfaz para crear objetos sin especificar exactamente sus clases.

Diagrama de Clases

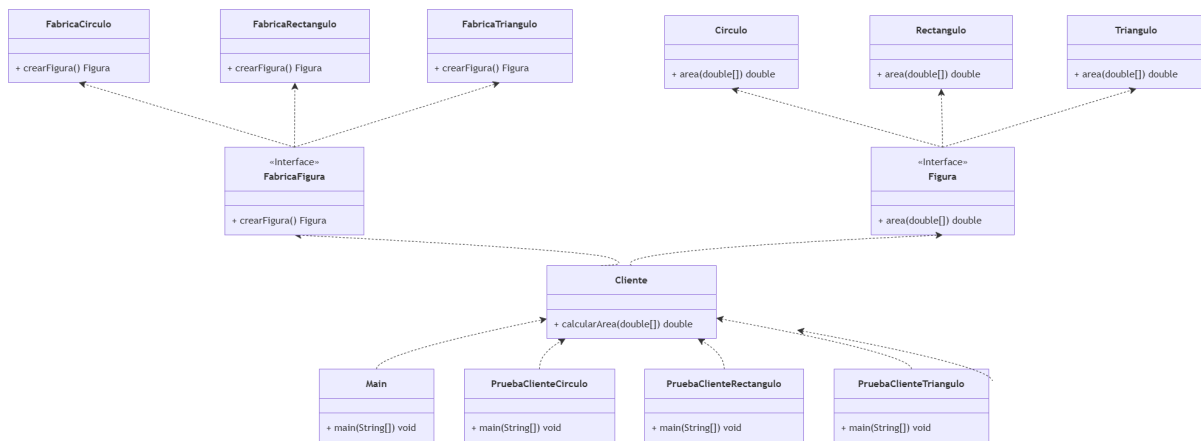


Figura 1: uml.

Explicación de las Clases

Producto - Figura

Descripción

La interfaz **Figura** es el contrato que definen todas las figuras geométricas del sistema. Establece que cualquier figura debe implementar el método **area()** para calcular su área.

Código Fuente

```
1 package figuras.fm;
2
3 /**
4  * @author natha
5  */
6 public interface Figura {
7
8     public double area(double ... parametro);
9
10 }
```

Explicación

- **Paquete:** `figuras.fm`
- **Método:** `area(double ... parametro)`
 - Utiliza parámetros variables (*varargs*)
 - Retorna un `double` con el área calculada
 - Permite diferentes números de parámetros según la figura
- **Propósito:** Define el comportamiento estándar para todas las figuras

Productos Concretos de Figuras

2.2.1. Clase Círculo

Código Fuente:

```
1 package figuras.fm;
2
3 /**
4  * @author natha
5  */
6 public class Circulo implements Figura{
7
8     @Override
9     public double area(double... parametro) {
10         return (parametro[0]*parametro[0]*Math.PI);
11     }
12
13 }
```

Explicación:

- **Implementa:** Interface Figura
- **Parámetros necesarios:** 1 (radio)
- **Fórmula:** $A = r^2 \times \pi$
- **Cálculo:**
 - Recibe el radio en parametro[0]
 - Multiplica el radio por sí mismo
 - Multiplica el resultado por π (Math.PI)
- **Retorna:** El área del círculo en unidades cuadradas

2.2.2. Clase Rectángulo

Código Fuente:

```
1 package figuras.fm;
2
3 /**
4  * @author natha
5  */
6 public class Rectangulo implements Figura{
7
8     @Override
9     public double area(double ... parametro) {
10         return parametro[0]*parametro[1];
11     }
12 }
```

12
13 }

Explicación:

- **Implementa:** Interface Figura
- **Parámetros necesarios:** 2 (base y altura)
- **Fórmula:** $A = base \times altura$
- **Cálculo:**
 - Recibe la base en parametro[0]
 - Recibe la altura en parametro[1]
 - Multiplica ambos valores directamente
- **Retorna:** El área del rectángulo en unidades cuadradas

2.2.3. Clase Triángulo

Código Fuente:

```
1 package figuras.fm;
2
3 /**
4  * @author natha
5  */
6 public class Triangulo implements Figura{
7
8     @Override
9     public double area(double... parametro) {
10
11         if (parametro.length == 2){
12             return (parametro[0]*parametro[1])/2;
13         }
14         else if(parametro.length == 3){
15             double semiper = (parametro[0]+parametro[1]+parametro
16                 [2])/2;
17             return Math.sqrt(semiper*(semiper-parametro[0])
18                 *(semiper-parametro[1])*(semiper-parametro[2]));
19         }
20         throw new IllegalArgumentException("");
21     }
22 }
```

Explicación:

- **Implementa:** Interface Figura

- **Parámetros:** Flexible (2 o 3 parámetros)
- **Dos métodos de cálculo:**
 1. **Con 2 parámetros** (base y altura):
 - Fórmula: $A = \frac{base \times altura}{2}$
 - Caso simple para triángulos rectángulos
 2. **Con 3 parámetros** (tres lados):
 - Usa la fórmula de Herón
 - Semiperímetro: $s = \frac{a+b+c}{2}$
 - Área: $A = \sqrt{s(s-a)(s-b)(s-c)}$
 - Válida para cualquier triángulo
- **Validación:** Lanza `IllegalArgumentException` si no se proporcionan 2 o 3 parámetros

Interfaz Factory - FabricaFigura

Descripción

La interfaz `FabricaFigura` define el patrón Factory Method, estableciendo el contrato que todas las fábricas concretas deben cumplir.

Código Fuente

```
1 package figuras.fm.Fabricas;
2
3 import figuras.fm.Figura;
4
5 /**
6  * @author natha
7  */
8 public interface FabricaFigura {
9
10     public Figura crearFigura();
11
12 }
```

Explicación

- **Paquete:** `figuras.fm.Fabricas`
- **Tipo:** Interface (Contrato del Patrón Factory)
- **Método:** `crearFigura()`
 - Sin parámetros
 - Retorna un objeto `Figura`
 - Cada fábrica concreta decide qué tipo de figura crear
- **Propósito:** Centralizar la creación de objetos `Figura`

Implementaciones

1. **FabricaCirculo:** Crea instancias de `Circulo`
2. **FabricaRectangulo:** Crea instancias de `Rectangulo`
3. **FabricaTriangulo:** Crea instancias de `Triangulo`

2.3.1. FabricaCirculo

Código Fuente:

```
1 package figuras.fm.Fabricas;
2
3 import figuras.fm.Circulo;
4 import figuras.fm.Figura;
5
6 /**
7  * @author natha
8  */
9 public class FabricaCirculo implements FabricaFigura{
10
11     @Override
12     public Figura crearFigura() {
13         return new Circulo();
14     }
15
16 }
```

Explicación:

- Implementa FabricaFigura
- Responsable de crear objetos Circulo
- Aísla la creación del objeto cliente

2.3.2. FabricaRectangulo

Código Fuente:

```
1 package figuras.fm.Fabricas;
2
3 import figuras.fm.Rectangulo;
4
5 /**
6  * @author natha
7  */
8 public class FabricaRectangulo implements FabricaFigura{
9
10     @Override
11     public Rectangulo crearFigura() {
12         return new Rectangulo();
13     }
14
15 }
```

Explicación:

- Implementa FabricaFigura

- Responsable de crear objetos `Rectangulo`
- Retorna tipo `Rectangulo` (covarianza)

2.3.3. FabricaTriangulo

Código Fuente:

```
1 package figuras.fm.Fabricas;  
2  
3 import figuras.fm.Figura;  
4 import figuras.fm.Triangulo;  
5  
6 /**  
7  * @author natha  
8  */  
9 public class FabricaTriangulo implements FabricaFigura{  
10  
11     @Override  
12     public Figura crearFigura() {  
13         return new Triangulo();  
14     }  
15  
16 }
```

Explicación:

- Implementa `FabricaFigura`
- Responsable de crear objetos `Triangulo`
- Retorna tipo `Figura` (polimorfismo)

Código de Clientes y Explicación

Clase Cliente

Código Fuente:

```
1 package figuras.fm;
2
3 import figuras.fm.Fabricas.FabricaFigura;
4
5 /**
6  * @author natha
7  */
8 public class Cliente {
9
10     private Figura objFigura;
11
12     public Cliente(FabricaFigura fabrica) {
13         this.objFigura = fabrica.crearFigura();
14     }
15
16     public double calcularArea (double ... parametros){
17         return objFigura.area(parametros);
18     }
19
20 }
```

Explicación:

- **Propósito:** Usar las figuras creadas por las fábricas
- **Atributo:** objFigura de tipo Figura
- **Constructor:**
 - Recibe una FabricaFigura como parámetro
 - Delega la creación a la fábrica
 - No necesita conocer la clase concreta
- **Método:** calcularArea(double ... parametros)
 - Delega al método area() de la figura
 - Retorna el área calculada

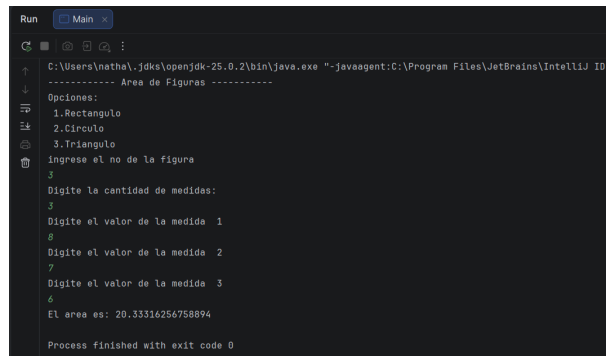
Main

Código Fuente Completo:

```
1 package figuras.fm;
2
3 import figuras.fm.Fabricas.FabricaCirculo;
4 import figuras.fm.Fabricas.FabricaFigura;
5 import figuras.fm.Fabricas.FabricaRectangulo;
6 import figuras.fm.Fabricas.FabricaTriangulo;
7 import java.util.Scanner;
8
9 /**
10  * @author natha
11  */
12 public class Main {
13
14     public static void main(String[] args) {
15
16         Scanner sc = new Scanner(System.in);
17         System.out.println("-----_Area_de_Figuras_
18             -----");
19         System.out.println("Opciones:_\n_1.Rectangulo_\n_2.
20             Circulo_\n_3.Triangulo");
21         System.out.println("ingrese_el_no_de_la_figura");
22         int opcion = sc.nextInt();
23
24         FabricaFigura fabrica;
25
26         if (opcion==1){
27             fabrica = new FabricaRectangulo();
28         }else if(opcion ==2){
29             fabrica = new FabricaCirculo();
30         }else if (opcion ==3){
31             fabrica = new FabricaTriangulo();
32         }else{
33             throw new IllegalArgumentException("Error");
34         }
35
36         Cliente objCliente = new Cliente(fabrica);
37
38         System.out.println("Digite_la_cantidad_de_medidas:_");
39         int num = sc.nextInt();
40
41         double [] parametros = new double[num];
42
43         for (int i=0; i<num; i++){
44             System.out.println("Digite_el_valor_de_la_medida_" +
45                 (i+1));
46             parametros[i] = sc.nextDouble();
47         }
48     }
49 }
```

```
46
47     double resultado = objCliente.calcularArea(parametros);
48
49     System.out.println("El area es: " + resultado);
50 }
51
52 }
```

Flujo de Ejecución



```
Run - Main
C:\Users\netho\.jdk\openjdk-25.0.2\bin\java.exe "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA
----- Area de Figuras -----
Opciones:
1.Rectangulo
2.Circulo
3.Triangulo
Ingrese el no de la figura
3
Digite la cantidad de medidas:
3
Digite el valor de la medida 1
8
Digite el valor de la medida 2
7
Digite el valor de la medida 3
6
El area es: 20.33316256758894
Process finished with exit code 0
```

Figura 2:

1. **Inicio:** Se solicita al usuario elegir una figura (1-Rectángulo, 2-Círculo, 3-Triángulo)
2. **Selección:** Según la opción, se crea la fábrica correspondiente
3. **Cliente:** Se instancia la clase Cliente con la fábrica
4. **Parámetros:** Se solicita el número de medidas y sus valores
5. **Cálculo:** El cliente calcula el área usando la figura
6. **Resultado:** Se muestra el área calculada

Pruebas

Prueba de Círculo

Código Fuente:

```
1 package figuras.fm.Pruebas;
2
3 import figuras.fm.Cliente;
4 import figuras.fm.Fabricas.FabricaCirculo;
5 import figuras.fm.Fabricas.FabricaFigura;
6 import java.util.Scanner;
7
8 /**
9  *
10  * @author natha
11  */
12 public class PruebaClienteCirculo {
13
14     public static void main(String[] Args){
15         FabricaFigura fabrica = new FabricaCirculo();
16         Cliente objCliente = new Cliente(fabrica);
17
18
19         double[] parametros=new double[1];
20         Scanner sc = new Scanner(System.in);
21         System.out.println("Ingrese el radio:");
22         parametros[0] = sc.nextDouble();
23
24         double resultado = objCliente.calcularArea(parametros);
25         System.out.println("El area del Círculo es: "+resultado);
26
27     }
28
29 }
```

Compilacion:

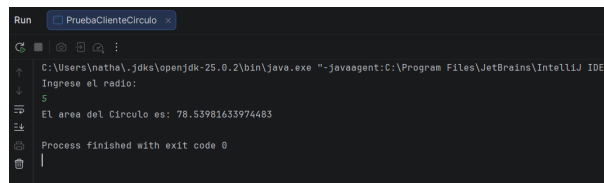


Figura 3:

Prueba de Rectángulo

Código Fuente:

```
1 package figuras.fm.Pruebas;
2
3 import figuras.fm.Cliente;
4 import figuras.fm.Fabricas.FabricaFigura;
5 import figuras.fm.Fabricas.FabricaRectangulo;
6 import java.util.Scanner;
7
8
9 /**
10  *
11  * @author natha
12  */
13 public class PruebaClienteRectangulo {
14
15
16     public static void main(String[] Args){
17         FabricaFigura fabrica = new FabricaRectangulo();
18         Cliente objCliente = new Cliente(fabrica);
19
20
21         double[] parametros=new double[2];
22         Scanner sc = new Scanner(System.in);
23         System.out.println("Ingrese la base:");
24         parametros[0] = sc.nextDouble();
25         System.out.println("Ingrese la altura:");
26         parametros[1] = sc.nextDouble();
27
28         double resultado = objCliente.calcularArea(parametros);
29         System.out.println("El area del Rectangulo es: "+
30             resultado);
31     }
32
33 }
```

Compilacion:



Figura 4:

Prueba de Triángulo

Código Fuente:

```
1 package figuras.fm.Pruebas;
2
3 import figuras.fm.Cliente;
4 import figuras.fm.Fabricas.FabricaFigura;
5 import figuras.fm.Fabricas.FabricaTriangulo;
6 import java.util.Scanner;
7
8 /**
9  *
10  * @author natha
11  */
12 public class PruebaClienteTriangulo {
13
14     public static void main(String[] Args){
15         FabricaFigura fabrica = new FabricaTriangulo();
16         Cliente objCliente = new Cliente(fabrica);
17
18
19         System.out.println("Digite la cantidad de medidas:");
20         Scanner sc = new Scanner(System.in);
21         int num = sc.nextInt();
22         double [] parametros = new double[num];
23
24         for (int i=0; i<num; i++){
25             System.out.println("Digite el valor de la medida "+
26                 (i+1));
27             parametros[i] = sc.nextDouble();
28         }
29
30         double resultado = objCliente.calcularArea(parametros);
31         System.out.println("El area del Triangulo es: "+resultado
32             );
33     }
34 }
35 }
```

Compilacion:



Figura 5:

Codigo

Link Github

- <https://github.com/Thalyhx/FactoryMethodFigures.git>

Paquetes

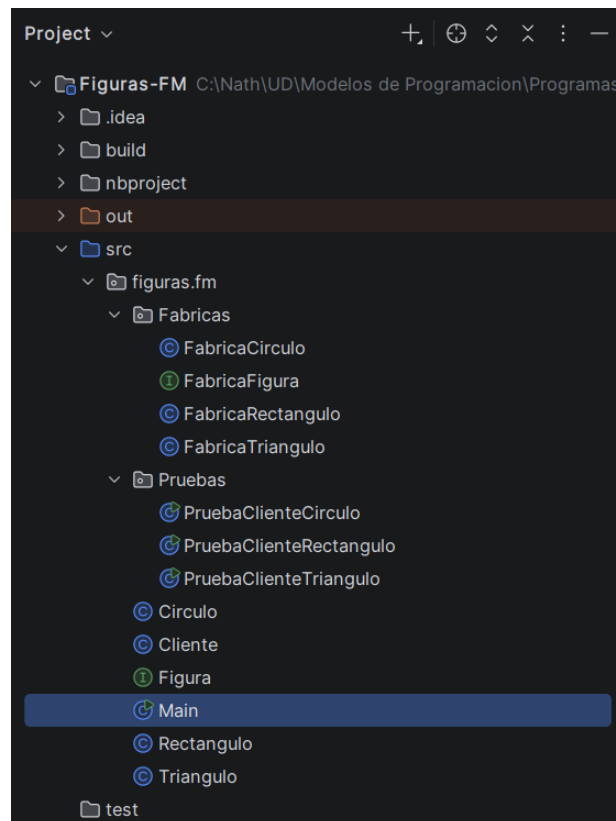


Figura 6: